

红黑树计数问题——算法设计、实现与性能分析

大家好！我是第九组mayuxi。我会从研究背景、算法策略、理论框架、数据分析和结论建议五个维度，带大家一步步理解这个经典组合数学问题的解决思路，也会穿插具体例子和细节，帮大家更直观地把握核心逻辑。

给定一个正整数 N （最多500个内部节点），要算出所有“结构不同”的红黑树总数。这里的“结构不同”很好理解，比如同样2个节点，根节点左边挂1个节点，和右边挂1个节点，就是两种不同结构。

那这个计数问题难在哪？如果 N 很小，比如 $N=1$ ，我们能直接看出只有1种；但 $N=100$ 、500时，直接枚举所有结构根本不可能——组合数量会呈指数级爆炸。所以我们的研究目的很明确：用动态规划解决这个计数问题。

我们设计的核心算法是动态规划，本质是紧扣红黑树的定义，把树的构建拆成子树的组合。

首先是子问题的定义：我们不直接算“ N 个节点的红黑树总数”，而是先算更细的子问题——有 h 个黑高度、 n 个内部节点，且根节点颜色为黑的红黑树有多少种，为红的红黑树有多少种。为什么要加黑高度 h 这个维度？因为红黑树的平衡性质，本质是通过黑高度控制的——只要黑高度固定，树的整体高度就不会失控，这也让子问题的拆分有了明确依据。

1. 如果根节点是黑色：根据红黑树性质，

- 它的左右子树的黑高度必须是 $h-1$ 。因为根自己占了一个黑节点。那要构建“ h 黑高度、 n 个节点、根黑”的树，就可以拆成：左子树分 i 个节点，右子树分 $n-1-i$ 个节点（因为根占1个），然后把“ $h-1$ 黑高度、 i 个节点的所有树”和“ $h-1$ 黑高度、 $n-1-i$ 个节点的所有树”组合起来——每一种左子树和右子树的搭配，都是一种新的树结构。

1. 如果根节点是红色：

- 性质要求它的左右子树必须是黑根（红节点的子节点不能是红）。
- 而且子树的黑高度和根的黑高度相同（因为根是红，不影响黑高度）。
- 所以拆分逻辑变成：左子树 i 个节点、右子树 $n-1-i$ 个节点，且左右子树都得是“ h 黑高度、黑根”的树，再把它们和红根组合。

比如 $N=2$ ，我们算根为黑的情况。此时 $n=2$ ，根占1个，左右子树共1个节点——要么左0右1，要么左1右0。

三、理论框架

首先是核心变量定义，我们用三个二维数组存储中间结果：

- $black[h][n]$: 黑高度 h 、 n 个节点、根为黑的红黑树数量;
- $red[h][n]$: 黑高度 h 、 n 个节点、根为红的红黑树数量;
- $total[h][n]$: 黑高度 h 、 n 个节点的红黑树总数 (即 $black[h][n] + red[h][n]$)。
- 外层循环“FOR $h=1..H$ ”: 遍历所有可能的黑高度 H 。这里 H 的上限怎么定? 其实红黑树的黑高度 h 满足 $h \leq \log_2(N+1)$, 比如 $N=500$ 时, $\log_2(501) \approx 9$, 所以 H 最大取10就够了, 不用无限制循环;
- 中层循环“FOR $n=1..N$ ”: 遍历1到 N 的所有节点数, 计算每个 n 对应的树数量;
- 内层循环“FOR $i=0..n-1$ ”: i 是左子树的节点数, 右子树节点数 j 就是 $n-1-i$ (因为根占1个);
- 计算 $black[h][n]$: 根为黑时, 左子树和右子树都是黑高度 $h-1$ 的任意树 (不管根色), 所以用 $total[h-1][i] * total[h-1][j]$, 累加所有 i 的情况;
- 计算 $red[h][n]$: 根为红时, 左子树和右子树必须是黑高度 h 且根为黑的树, 所以用 $black[h][i] * black[h][j]$, 同样累加所有 i 的情况。

举个具体的计算例子: 比如算 $n=3$ 、 $h=2$ 的 $black[2][3]$ 。此时 i 可以取0、1、2:

- $i=0, j=2$: $total[1][0]$ (空树, 算1种) * $total[1][2]$ ($h=1$ 、 $n=2$ 的总数, 假设是2) =2;
- $i=1, j=1$: $total[1][1]$ (1种) * $total[1][1]$ (1种) =1;
- $i=2, j=0$: 和 $i=0$ 一样, 得2;
- 所以 $black[2][3] = 2+1+2=5$? 不对, 这里要注意 $h=1$ 时的 $total$ 值, 其实 $h=1$ 时, 红黑树的黑高度是1, 根必须是黑 (因为 $h=1$, 根是唯一的黑节点), 所以 $total[1][n]$ 其实就是 $black[1][n]$, 而 $n=1$ 时 $total[1][1]=1$, $n=2$ 时 $total[1][2]$ 其实是0 (因为 $h=1$ 时, 根黑, 子树黑高度得是0, 只能是空树, 所以 $n=2$ 无法满足 $h=1$) ——这也能看出, 伪代码的循环是严格遵循红黑树性质的, 不会出现不符合逻辑的计算。

最后是复杂度分析: 时间上, 三层循环 HMN , 所以是 $O(HN^2)$; 空间上, 三个二维数组都是 HN 大小, 所以是 $O(H*N)$ 。这里 H 很小 ($N=500$ 时 $H \approx 10$), 所以实际复杂度接近 $O(N^2)$, 对 $N=500$ 来说, 计算量完全可控。

四、数据分析

1. **最小非空树 ($N=1$)**: 预期结果1种——只有1个黑节点的树, 实际输出也是1, 通过;
2. **小型倾斜树 ($N=2$)**: 预期2种——根黑, 左子树1个红节点; 根黑, 右子树1个红节点 (因为根红的话, 子树得是黑根且 h 相同, 但 $n=2$ 时根红的话, 子树各0个节点, 其实也是1种? 不对, 这里要注意 $N=2$ 时根红的情况: 根红, 左右子树都是黑根、 $h=h$ (根的 h), 但子树节点数0, 所以红根的树有1种, 加上黑根的1种, 总共2种, 和测试结果一致);
3. **V型/链型树 ($N=3$)**: 其实这里的“2种”是指结构上的典型情况——比如根黑, 左右各1个红节点 (V型); 根黑, 左子树黑、左子树的左子树红 (链型), 实际计算结果也是2, 通过;
4. **样本案例 ($N=5$)**: 参考问题中的标准样本, 预期8种, 实际输出8, 通过;
5. **中等规模 ($N=100$) 和最大规模 ($N=500$)**: 这两组是关键, 尤其是 $N=500$ 时, 结果要模 $1e9+7$ 预期167844408和874880073, 实际输出完全一致, 通过。

五、结论与建议：优化与未来方向（约1分钟）

首先是**核心结论**：我们设计的直接DP算法能正确解决红黑树计数问题，时间复杂度 $O(HN^2)$ ，空间复杂度 $O(HN)$ 。

然后是**空间优化**：我们发现黑高度 h 的计算是“顺序依赖”的——算 h 时只需要 $h-1$ 的数据，所以可以用“奇偶索引滚动”优化：用 $h \& 1$ （0或1）表示当前 h 的索引， $(h-1) \& 1$ 表示前一个 h 的索引，这样数组从二维变成一维（比如`black[current][n]`，`current`是0或1），空间复杂度直接从 $O(H \cdot N)$ 降到 $O(N)$ ，比如说， $N=500$ 时，内存占用从原来的4500个单元（ $H=9$ ）变成500个，效率提升很明显。

以上就是我今天的全部汇报内容，讲解过程中如果有逻辑不够清晰的地方，或者大家有更好的优化思路，欢迎随时指出——恳请各位老师、同学批评指正！谢谢大家！